Reinforcement Learning for Dynamic Data Fragmentation and Allocation

Midterm Draft

Tino Kaseke
Department of Computer Science
University of Texas Permian Basin
Odessa, Texas, USA
kaseke_t87479@utpb.edu

***Abstract*—Distributed databases decide how to split their data and where to put the pieces once, when the database is designed, using a function of the key that never sees where requests come from. When the geographic centre of a workload moves, as it does daily in a service spread across regions, that placement drifts out of step with the traffic and transactions get slower. This paper states continuous re-splitting and re-placement as a reinforcement learning problem over a simulated six-region cluster. On a real six-node CockroachDB cluster, gathering primary copies in the region issuing the transactions rather than spreading them changes 95th percentile latency by 1372.5 ms, so the premise holds outside simulation. Sweeping how fast the workload moves shows a simple reactive rule 0.44 ms above the best available placement under slow drift and 57.20 ms above it under fast drift, which is 41% of its own latency; at that setting the rule stops beating fixed placement at all once movement and imbalance are counted. A Deep Q-Network trained there beats both baselines on held-out seeds, on two training seeds, by moving one twentieth of the data the rule moves. The margin is narrow and the policy had not converged, so this is evidence that the room the sweep identifies is reachable, not that the problem is solved.**

***Index Terms*—distributed databases, data fragmentation, data allocation, reinforcement learning, deep Q-networks, geo-distributed transactions**

I. Introduction

A distributed database splits its data into pieces and spreads those pieces across machines in different places. The pieces are called shards. Two decisions matter. The first is where the boundaries between shards fall. The second is which machine holds each shard. Both are normally made once, when the database is designed, and the placement is then decided by a function of the key, such as a hash. A hash does not know where requests come from.

That is fine whilst the traffic stays where the designer expected. It is not fine in a service with users on several continents, served by a database spread across regions in the manner of Spanner [1] or CockroachDB [2]. The geographic centre of the traffic moves across the day as one region wakes and another sleeps, so a placement chosen for the morning is wrong by the evening. Every request that crosses an ocean to reach its data pays that ocean's round-trip time, and a transaction touching shards in several regions pays it more than once, because keeping such a transaction atomic needs more than one round of agreement between the machines holding those shards.

The obvious response is to move the data when the traffic moves. A simple rule does this: every decision window, find the shard whose placement is costing the most and move it towards the region reading it. I implement that rule as a baseline and call it *follow traffic*.

This paper asks whether a policy learned from experience does better than that rule. The answer turns out to depend on how fast the traffic moves. When the drift is slow, the simple rule is within half a millisecond of the best placement that moving shards can reach, so there is nothing worth learning. When the drift is fast, the same rule falls 57 ms short of that reference, which is 41% of its own latency, because it always reacts one window behind a target that has already moved again. The room for a learned policy is largest exactly where the problem is hardest.

The paper builds a simulator of a six-region cluster whose traffic drifts on a daily cycle and spikes locally, states placement on it as a reinforcement learning problem with 261 possible actions, and measures four reference points against it: fixed hash placement, random movement, the follow traffic rule, and a cost-free reference showing what good placement is worth when movement is not charged for. It checks on a real six-node CockroachDB cluster that placement changes measured latency by far more than the measurement noise. It then trains a Deep Q-Network at the fastest drift setting, where it beats both the rule and fixed placement on the combined cost.

II. Related Work

Three bodies of work meet in this problem: the classical treatment of fragmentation and allocation as a problem to be solved once, modern systems that move data in response to the traffic they see, and the use of learning to replace hand-written database administration.

The classical treatment is old and well understood. Fragmentation was studied as vertical [3] and horizontal partitioning of a schema, and allocation as an optimization problem over a known workload. The assumption running through all of it is that the workload is known in advance and does not move. Production systems make the same assumption in a cheaper form: consistent hashing [4] spreads keys evenly and ignores geography entirely.

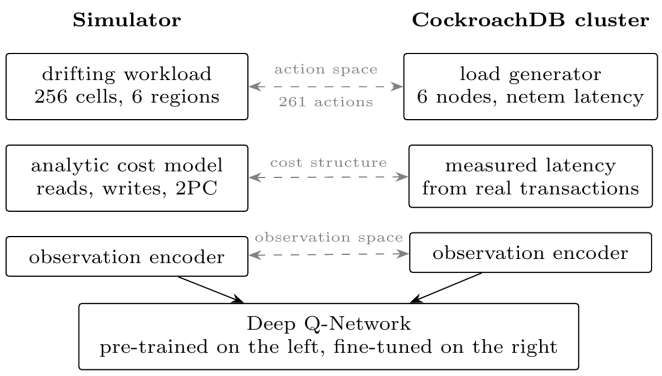


Fig. 1. The two halves of the system and the three things that must match between them. The agent is trained against the simulator, which is fast and cheap, and is intended to be fine-tuned against the cluster, which is slow and real. Training in one and running in the other is only meaningful if the observations, the actions and the way cost is charged are the same on both sides.

Systems that react to the workload came later. Schism [5] builds a graph of which records are accessed together and partitions it to cut distributed transactions. Clay [6] detects hot records and moves them, which addresses skew within a cluster rather than drift between regions. Lion [7] provisions replicas adaptively using a cost model. All of these react to what has already happened. None of them learns a policy.

The learning side has grown quickly, mostly in tuning rather than placement. Sibyl [8] learns data placement in a storage hierarchy, and reinforcement learning has been applied to database configuration tuning. The closest published work is RLRP [9], which uses reinforcement learning for data placement and migration in heterogeneous distributed storage. RLRP places objects as demand and capacity change. This project differs in what the agent is allowed to decide: it may also split a shard, it is judged on transactions that span several shards rather than on single object access, and the thing it is tracking is geography rather than capacity.

The methods used here are standard. The agent is a Deep Q-Network [10] trained with Stable-Baselines3 [11], and the structure of training in a cheap simulator before fine-tuning on expensive real hardware is borrowed from robotics. The contribution is therefore the measurement rather than the machinery: what is new is a statement of when a learned policy is worth having, with the condition measured rather than assumed.

III. System Design

A. The simulator

The simulated cluster has six regions, one node in each, and three copies of every shard. The keyspace is divided into 256 cells, and shards are built from runs of those cells. An episode is 200 decision windows and the simulated day is 96 windows, so one episode covers about two days.

Traffic comes from a centre of demand that rotates around the regions on that daily cycle, with local spikes on top. A single number, *volatility*, scales how fast the centre rotates and how often and how hard it spikes, so one setting moves the environment from placid to violent. It is the main thing the experiments vary.

TABLE I
Simulator and training configuration

Item	Setting
Regions	6, one node each, 3 copies per shard
Keyspace	256 cells, 6 starting shards, 20 shard cap, 60 GB
Episode	200 decision windows; 96-window day
Per window	8,000 single-key requests, 2,000 three-key transactions, 80% reads
Agent	Deep Q-Network, two 256-unit layers, 261 actions
Training	300,000 to 1,000,000 steps, 64 seeds
Seeds	sweep 0 to 7; agent 2000 to 2007, held out
Cluster	6 CockroachDB nodes, delays set with netem

B. What the agent sees and does

At each decision window the agent is shown the read and write counts for each region from the window just finished, where every shard currently sits, how big each shard is, how much data has moved recently, and the matrix of round-trip times between regions.

It then picks one action from 261. There are four kinds. MOVE_LEASE makes a region the primary copy for a shard, which moves no data and is only legal where that region already holds a copy. MOVE_REPLICA_TO puts a copy of a shard in a region, dropping the existing copy furthest away so the number of copies stays the same, and this one does move data. SPLIT divides a shard at its midpoint. NOOP does nothing.

Every action names a desired end state rather than an operation, so every action is defined in every state, and one that cannot be applied resolves to a small penalty instead of an error. This matters because the kind of network used here cannot be told which actions are illegal.

C. What the agent is scored on

The agent minimises one combined cost,

$$c = \frac{\ell}{100} \; + \; w \cdot \frac{g}{2000} \; + \; 0.10 \cdot b \tag{1}$$

where ℓ is mean transaction latency in milliseconds, g is gigabytes moved over the episode, b is load imbalance, and w sets how much movement is charged for. In plain terms: latency is the thing to make small, moving data is charged for, and piling the whole workload onto one node is charged for separately, so a policy cannot win on latency by overloading one region.

Movement is charged as a fraction of the dataset rather than in bytes. This is deliberate. A 60 GB simulator and a 1 GB cluster then score the same physical move identically, so a policy trained in one can be scored in the other without the reward changing meaning underneath it.

IV. Experimental Method

A. What is compared

Four reference points are measured against the agent. *Fixed placement* assigns shards round-robin at the start and never changes, which is what production systems do today. *Random* moves shards at the same rate as the heuristic but picks destinations without looking at the traffic. *Follow traffic* is the reactive rule from Section I. The *cost-free reference* independently picks the best placement for the current window and is not charged for movement at all.

The reference is easy to overread. It keeps the shard boundaries the episode started with, so it is not a global best, and a policy that splits a shard can legitimately score below it. Every policy is driven by the same episode loop, the learned one included, so a difference between two numbers in this paper cannot be a difference between two pieces of harness.

B. Seeds and reporting

Two separate seed blocks appear, and they are never mixed in one table. The volatility sweep uses seeds 0 to 7. The agent evaluations use held-out seeds 2000 to 2007, which the training configuration refuses to overlap with its training seeds. A sweep number and an agent number are therefore not directly comparable.

Results are the mean over eight seeds. Every run writes one self-contained directory holding the resolved configuration, the seed list, the commit, the package versions, the machine, the raw per-window record and the command that reproduces it, and runs execute on hosted continuous integration rather than on a personal machine.

C. The real cluster

The simulator's cost model is an assumption, and the assumption is testable. I deploy six CockroachDB nodes in containers on one host, one per simulated region, with wide-area round-trip times injected between them by the Linux kernel's traffic control facility. CockroachDB was chosen on one criterion: it offers a statement that relocates a shard's primary copy and does not return until the move has actually happened. If a placement command returned early, the cost measured after an action would belong to a state the action did not produce, and the decision problem would stop being well defined.

V. Preliminary Results

A. Placement changes latency on real hardware

Table II reports two pilots in which the same transactions run twice, once with the primary copies gathered in the region issuing them and once with them spread across regions. The difference in 95th percentile latency is 1372.5 ms in the first pilot and 1202.0 ms in the second. Placement is worth a great deal on real hardware, which is the assumption everything else rests on.

Three qualifications belong with that number. First, co-located means the primary copies, not the data. With three copies and one node per region the copies always span three regions, so every write still pays a cross-region agreement even in the co-located case, which is why that median is 272.7 ms rather than single digits. The experiment isolates the effect of where the primary sits, which is what MOVE_LEASE controls.

TABLE II
Cluster pilots. Transaction latency with primary copies gathered in the region issuing the transactions, against spread across regions.

Pilot	Placement	p50 (ms)	p95 (ms)	stdev (ms)
$N = 200$, 2 cores	co-located	272.7	275.5	48.8
	spread	1013.4	1648.0	302.0
$N = 1000$, 4 cores	co-located	441.5	446.6	78.4
	spread	1015.3	1648.7	299.9

TABLE III
Volatility sweep. Mean latency in milliseconds and gigabytes moved per episode, eight seeds per value, seed block 0 to 7.

Volatility	Fixed	Random	Follow	Reference	Follow GB
0.5	156.27	156.79	94.31	93.87	405.1
1.0	156.11	158.02	98.66	93.66	785.2
2.0	155.37	158.89	125.37	89.47	1056.6
4.0	155.77	160.24	139.58	82.38	1115.2

Second, the figures are indicative rather than precise. At 200 transactions the 95th percentile is the 190th value in the sorted list, so an interval around it is unreliable. The second pilot raised the count to 1000 and halved the interval width from 155.2 to 65.6 ms.

Third, the co-located case became 62% slower when the pilot moved from two cores to four, whilst the spread case barely moved. The hardware and the transaction count changed together, so this isolates no cause. It is consistent with every primary landing on one node and that node becoming the limit, which the simulator cannot reproduce, and it is why imbalance is charged for separately in (1).

B. How the room for a better policy changes

The random baseline separates two explanations that are easy to confuse. It moves data at the same rate as the rule but picks destinations without looking at the traffic, and at every volatility in Table III it moves 204.8 GB to land between 156.79 and 160.24 ms, no better than doing nothing. Movement by itself buys nothing. All of the gain is in the choice of destination, which is why this is a decision problem at all.

Table III and Fig. 2 then sweep volatility over a factor of eight, and three things happen at once.

Fixed placement is flat, 155.37 to 156.27 ms, because it was never tracking the traffic in the first place.

The cost-free reference gets *better*, 93.87 down to 82.38 ms. This is worth a sentence because it reads backwards at first. Faster drift means the traffic is more sharply concentrated at any one moment, and sharply concentrated traffic is easier to

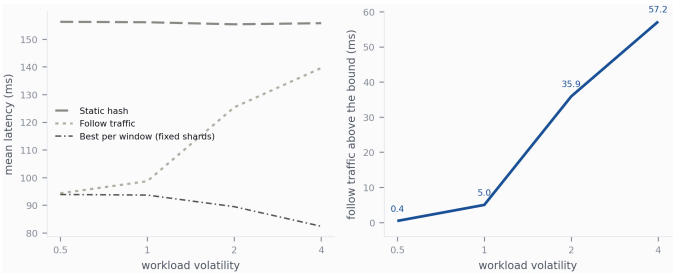


Fig. 2. Left: latency as the traffic moves faster, for fixed placement, the follow traffic rule, and the cost-free reference. Right: how far the rule sits above that reference. That distance is the room available to a better policy, and it opens from 0.44 ms to 57.20 ms.

TABLE IV
LEARNED POLICY AGAINST THE BASELINES ON HELD-OUT SEEDS 2000 TO 2007. COSTS ARE FROM (1), EACH PAIR SCORED AT THE SAME w. NEGATIVE MEANS THE AGENT IS BETTER.

Run	Vol	w	Seed	ms	GB	vs follow	vs fixed
3	4.0	0.30	0	139.91	62.7	−0.068	−0.071
7	4.0	0.30	1	144.61	44.5	−0.031	−0.035
4	4.0	0.10	0	136.94	123.7	+0.012	−0.100
8	2.0	0.30	0	129.64	177.3	+0.003	−0.167
6	2.0	0.00	0	129.24	311.9	+0.124	−0.202

serve well if the data is in the right place. So faster drift makes the problem harder for a policy and easier for an oracle.

The rule goes the other way. Its latency rises from 94.31 to 139.58 ms whilst the data it moves rises from 405.1 to 1115.2 GB. It pays nearly three times as much and gets progressively less for it, because a rule that reacts to the window just finished is chasing a target that has already moved on.

The result is the right-hand panel of Fig. 2. The distance between the rule and the reference is 0.44 ms at the slowest setting and 57.20 ms at the fastest, which is 41% of the rule's own latency. Under slow drift a reactive rule is already close to the best available and there is nothing worth learning. Under fast drift there is a great deal worth learning.

C. The learned policy

Table IV reports five agent runs on held-out seeds, each scored against the baselines at that run's own movement weight. Scoring a learned run against a baseline measured at a different weight compares a policy with a differently judged opponent, and it is the easiest way to manufacture a win, so the arithmetic is produced by a script rather than by hand.

The agent beats fixed placement in all five runs. At the fastest drift setting with $w = 0.30$ it also beats the rule, on both training seeds, by 0.068 and 0.031 of combined cost. The way it does so is easy to describe. It reaches 139.91 ms against the rule's 129.04 ms, so it is slower, but it moves 62.7 GB against 1086.5 GB and holds imbalance at 0.776 against 1.003. It is buying its latency far more cheaply than the rule buys its own.

Fig. 3 shows a second result from the same scoring. At the fastest setting the rule and fixed placement are indistinguishable on the combined cost, 1.554 against 1.557, even

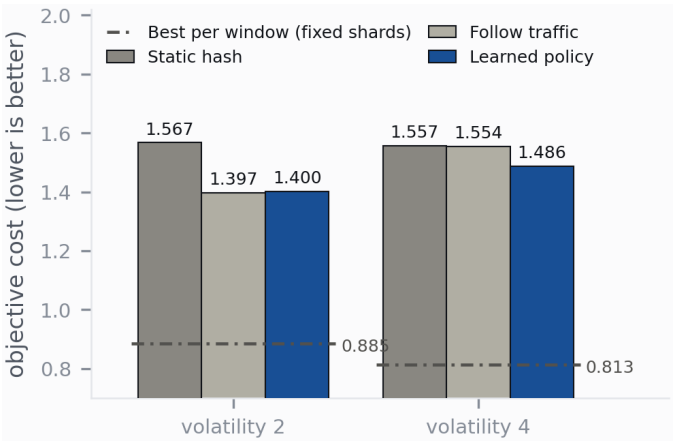


Fig. 3. Every policy scored on the combined cost at $w = 0.30$, not on latency alone. At the fastest drift setting the rule's latency advantage is almost entirely paid back by the data it moves and the imbalance it leaves, and the learned policy is below both baselines. The distance still left to the reference is the honest measure of how much remains to do.

though the rule is 21 ms faster. Everything it gains on latency it hands back in movement and imbalance. By that point the reactive rule has stopped being worth running, which is a sharper version of the previous subsection's finding than the latency curve alone can give.

VI. DISCUSSION

The measurements support one claim, and it is narrower than the project's original question. Whether a learned policy is worth having depends on how fast the workload moves, and that condition is now measured rather than assumed. A learned policy is worth building under fast drift and is a waste of effort under slow drift.

The win over the rule is real but narrow, and it is a win over an opponent that has itself stopped working. The agent reaches it by being slower and much cheaper, which is what the combined cost was written to reward, so this is the objective working as intended rather than a surprise. It is not evidence that the agent has learned to place data well. The cost-free reference sits at 0.813 against the agent's 1.486, and that distance, not the distance to the rule, is the measure of what is left.

One methodological point generalises beyond this project. Comparing a learned policy with a heuristic on latency alone compares them on something only one of them is optimising. Reporting latency alone would make the agent look worse than it is, and reporting movement alone would make it look better. Report the two together, or report the combined cost.

VII. LIMITATIONS AND REMAINING WORK

The simulator's cost model is a set of formulas over synthetic demand, without protocol, storage or queueing delays. The cluster pilots establish that placement matters on real hardware; they do not establish that the simulator's numbers are right. The two pilots also changed the transaction count and the core count together, so the difference between them

cannot be attributed to either alone, and that count was chosen by convenience rather than by a power calculation.

Three limitations sit on the learning side and they are the substantial ones. No run had converged: held-out evaluation was still improving at the final training step in every case, so the reported numbers are a floor rather than a ceiling. Nothing can yet be claimed about the movement weight, because the spread between two training seeds at the fastest setting is 4.70 ms whilst changing the weight from 0.30 to 0.10 at a fixed seed changes latency by 2.97 ms, so the difference between seeds is larger than the effect being studied. And between 79.9% and 89.8% of the actions the agent selects are inapplicable in the state it selected them in, against 80.4% for the random baseline, so it is winning on the minority of its decisions that land.

That last figure is the largest measured inefficiency in the project and the first thing to fix. It is a known failure mode as the illegal share of an action list grows [12], and action-elimination networks built on this kind of agent address it directly [13]. After that, in order: more seeds per setting, which is cheap and unblocks the movement-weight question; better credit assignment, since a move is charged in full when it is made whilst the latency it buys arrives over later windows, which n-step returns [14] would address; longer training; separating the transaction count from the core count; building the harness that would let the trained agent act on the real cluster, which does not exist yet; and adding a cost-model rebalancer in the style of Lion [7] as a stronger opponent.

VIII. Conclusion

This paper tested whether a reinforcement learning agent can act as an autonomous database administrator, moving shards in real time as the geographic centre of a workload moves. Three things are established. Placement changes 95th percentile latency on a real six-node cluster by 1372.5 ms, so the premise holds outside simulation. The room available to a better policy grows with how fast the workload moves, from 0.44 ms under slow drift to 57.20 ms under fast drift, and at the fastest setting the reactive rule stops beating fixed placement on the combined cost at all. And a Deep Q-Network trained at that setting beats both baselines on held-out seeds, on two independent training seeds, by moving one twentieth of the data the rule moves.

The win is narrow, over an opponent that has itself stopped working, with a policy that had not converged and that wastes most of its decisions on inapplicable actions. It should be read as evidence that the room identified by the sweep is reachable, not as a solved problem. The remaining distance to the cost-free reference is larger than the distance already closed.

Disclosure of AI Use

Claude, by Anthropic, was used throughout this project as a coding and writing assistant. It helped implement the simulator, the baselines and the experiment scripts, drafted and revised sections of this paper, and checked the arithmetic in the result tables. All experiments were specified, dispatched and reviewed by the author. Every number reported here was transcribed from a recorded run directory, each recorded row names the run it came from, and the cost arithmetic in Table IV is reproduced by a script in the project repository rather than computed by hand. The author is responsible for the content and the claims made.

References

[1] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013.

[2] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, "CockroachDB: The resilient geo-distributed SQL database," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1493–1509, 2020.

[3] S. Navathe, S. Ceri, G. Wiederhold, and J. Dou, "Vertical partitioning algorithms for database design," *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 680–710, 1984.

[4] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the world wide web," in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing (STOC)*, pp. 654–663, 1997.

[5] C. Curino, E. Jones, Y. Zhang, and S. Madden, "Schism: a workload-driven approach to database replication and partitioning," *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 48–57, 2010.

[6] M. Serafini, R. Taft, A. J. Elmore, A. Pavlo, A. Aboulnaga, and M. Stonebraker, "Clay: Fine-grained adaptive partitioning for general database schemas," *Proceedings of the VLDB Endowment*, vol. 10, no. 4, pp. 445–456, 2016.

[7] Q. Zheng, Z. Zhao, W. Lu, C. Yao, Y. Chen, A. Pan, and X. Du, "Lion: Minimizing distributed transactions through adaptive replica provision," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pp. 2012–2025, 2024.

[8] G. Singh, R. Nadig, J. Park, R. Bera, N. Hajinazar, D. Novo, J. Gómez-Luna, S. Stuijk, H. Corporaal, and O. Mutlu, "Sibyl: adaptive and extensible data placement in hybrid storage systems using online reinforcement learning," in *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pp. 320–336, 2022.

[9] K. Lu, N. Zhao, J. Wan, C. Fei, W. Zhao, and T. Deng, "RLRP: High-efficient data placement with reinforcement learning for modern distributed storage systems," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 595–605, 2022.

[10] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[11] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[12] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," *The International FLAIRS Conference Proceedings*, vol. 35, 2022.

[13] T. Zahavy, M. Haroush, N. Merlis, D. J. Mankowitz, and S. Mannor, "Learn what not to learn: Action elimination with deep reinforcement learning." arXiv:1809.02121, 2018.

[14] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.